🔟 Aggregation Functions

(Conceptual + Practical First — Professional depth later)

🔹 PART 1 — What Is Aggregation?

Before learning any function, understand this:

Aggregation = reducing many values into fewer values.

It **summarizes data**.

Example:

```
[10, 20, 30, 40]
```

Aggregation could give:

- Total → 100
- Average → 25
- Minimum → 10
- Maximum → 40

So instead of keeping all 4 numbers, we reduce them to 1 number.

In NumPy, aggregation works on:

- Entire arrays
- Rows
- Columns
- Higher dimensions

And this is controlled by **axis** (very important — we'll master it properly).

🔹 1️⃣ np.sum()

Concept

Adds values together.

Mathematically:

$$\text{sum} = x_1 + x_2 + x_3 + \ldots + x_n$$

Practical (1D)

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(np.sum(arr))
```

Output:

```
10
```

It collapsed 4 values into 1 value.

Practical (2D)

```
X = np.array([[1,2,3],
              [4,5,6]])

print(np.sum(X))
```

Output:

```
21
```

All elements added together.

Now we introduce something critical.

🔥 AXIS (Start Understanding Slowly)

Look at this matrix:

```
X = np.array([[1,2,3],
              [4,5,6]])
```

Shape = (2 rows, 3 columns)

Visually:

```
Row 0 → [1 2 3]
Row 1 → [4 5 6]
```

🔹 axis = 0

```
np.sum(X, axis=0)
```

Think: “Combine downward ↓”

Column-wise addition:

```
1 + 4 = 5
2 + 5 = 7
3 + 6 = 9
```

Result:

```
[5 7 9]
```

It collapsed rows.

🔹 axis = 1

```
np.sum(X, axis=1)
```

Think: “Combine across →”

Row-wise addition:

```
1 + 2 + 3 = 6
4 + 5 + 6 = 15
```

Result:

```
[6 15]
```

It collapsed columns.

🧠 Very Important Rule

Axis tells NumPy:

“Which dimension should I reduce?”

In 2D:

- axis=0 → reduce rows
- axis=1 → reduce columns

We'll master this deeply as we move forward.

🔹 2️⃣ np.mean()

Concept

Mean = average

$$\text{mean} = \frac{\text{sum}}{\text{count}}$$

1D Example

```
arr = np.array([10, 20, 30])
print(np.mean(arr))
```

Output:

```
20.0
```

Notice:
Result is float.

Mean ALWAYS returns float.

2D with Axis

```
X = np.array([[1,2,3],
              [4,5,6]])

print(np.mean(X, axis=0))
```

Column averages:

```
(1+4)/2 = 2.5
(2+5)/2 = 3.5
(3+6)/2 = 4.5
```

Output:

```
[2.5 3.5 4.5]
```

Why This Is Important

In Machine Learning:

```
mean = np.mean(X, axis=0)
X_centered = X - mean
```

This is feature normalization.

3 np.min() and np.max()

Concept

Minimum → smallest value
Maximum → largest value

Example

```
arr = np.array([5, 2, 9, 1])

print(np.min(arr))  # 1
print(np.max(arr))  # 9
```

With axis

```
X = np.array([[1,2,3],
              [4,5,6]])

print(np.min(X, axis=0))
print(np.max(X, axis=1))
```

Understand what you expect before running.

That's mastery.

🔹 4️⃣ np.argmin() and np.argmax()

VERY IMPORTANT difference.

- max → returns value
- argmax → returns position (index)

Example:

```
arr = np.array([10, 50, 30])

print(np.max(arr))      # 50
print(np.argmax(arr))   # 1
```

Because 50 is at index 1.

In ML

```
predictions = np.array([0.1, 0.7, 0.2])
print(np.argmax(predictions))
```

This gives predicted class index.

Used everywhere in classification.

🔹 5️⃣ np.var() and np.std()

These measure spread of data.

Variance

$$\frac{1}{n}\sum(x_i - \mu)^2$$

Measures squared spread.

Standard Deviation

$$\sqrt{\text{variance}}$$

Gives spread in original units.

Example:

```
arr = np.array([1,2,3,4])

print(np.var(arr))
print(np.std(arr))
```

With axis

```
X = np.array([[1,2,3],
              [4,5,6]])

print(np.std(X, axis=0))
```

Gives column-wise standard deviation.

Now Pause — Let's Strengthen Axis Deeply

Axis is the most important concept here.

Remember:

If shape is:

```
(rows, columns)
```

Then:

axis=0 → go down ↓
axis=1 → go right →

Another way to think:

Axis number = dimension being removed.

Example:

Shape (2,3)

After:

```
np.sum(X, axis=0)
```

Result shape → (3,)

After:

```
np.sum(X, axis=1)
```

Result shape → (2,)

Axis dimension disappears.

Professional Phase Start Here:

2 FULL Axis Mastery

If:

```
X.shape = (2, 3, 4)
```

You have 3 axes:

- axis 0 → depth
- axis 1 → rows
- axis 2 → columns

You can reduce:

```
np.sum(X, axis=2)  # reduce last dimension
np.sum(X, axis=(1,2))  # reduce multiple axes
```

Multi-axis reduction:

```
np.sum(X, axis=(1,2))
```

Reduces both axes simultaneously.

Professionals use this constantly.

1️⃣ What is NaN?

NaN = **Not a Number**

In NumPy:

```
np.nan
```

It represents:

- Missing data
- Undefined results
- Corrupted entries

Important:

```
np.nan == np.nan
```

Returns:

```
False
```

Because NaN is not equal to anything — even itself.

🔥 2️⃣ The Problem With Normal Aggregations

Example:

```
import numpy as np

X = np.array([10, 20, np.nan, 40])

np.mean(X)
```

Result:

```
nan
```

Why?

Because arithmetic with NaN propagates:

$$10 + 20 + NaN + 40 = NaN$$

So:

One NaN contaminates the entire aggregation.

This is called **NaN propagation**.

🔥 3️⃣ Why This Is Dangerous

Imagine a dataset:

- 10,000 values
- 1 missing value

Normal mean:

```
np.mean(X)
```

→ Entire result becomes NaN
→ Your ML pipeline crashes
→ Your metrics become invalid

This happens constantly in real datasets.

🔥 4️⃣ The Professional Solution: NaN-Aware Functions

NumPy provides special versions:

Normal	NaN-aware
`np.sum`	`np.nansum`
`np.mean`	`np.nanmean`
`np.std`	`np.nanstd`
`np.var`	`np.nanvar`
`np.min`	`np.nanmin`
`np.max`	`np.nanmax`

These **ignore NaN values**.

🔥 5️⃣ Example: `np.nanmean`

```
X = np.array([10, 20, np.nan, 40])

np.nanmean(X)
```

Now result:

```
23.3333
```

Because it computes:

$$\frac{10 + 20 + 40}{3}$$

NaN is skipped.

🔥 6️⃣ Axis with NaN-aware Aggregations

Works exactly like normal aggregation.

Example:

```
X = np.array([[1, 2, np.nan],
              [4, np.nan, 6]])

np.nanmean(X, axis=0)
```

Column-wise mean ignoring NaNs:

Column 1 → (1 + 4)/2 = 2.5
Column 2 → 2/1 = 2
Column 3 → 6/1 = 6

Result:

```
[2.5, 2.0, 6.0]
```

🔥 7️⃣ What If Entire Slice Is NaN?

Example:

```
X = np.array([np.nan, np.nan])

np.nanmean(X)
```

Result:

```
nan
```

And warning:

```
RuntimeWarning: Mean of empty slice
```

Because after ignoring NaNs, nothing remains.

Professional takeaway:

NaN-aware functions ignore NaNs — but cannot invent data.

🔥 8️⃣ How NumPy Implements This Internally

Conceptually:

```
valid = X[~np.isnan(X)]
mean = np.sum(valid) / len(valid)
```

But implemented efficiently in C.

It:

- Counts non-NaN values
- Accumulates only valid values
- Divides by valid count

🔥 9️⃣ Why This Is Critical in ML

Real-world data contains:

- Missing sensor values
- Incomplete surveys
- Database nulls
- Broken CSV imports
- Financial gaps

Example:

```
mean = np.nanmean(X, axis=0, keepdims=True)
X_centered = X - mean
```

If you used `np.mean`, your entire normalization would become NaN.

🔟 Weighted Aggregations (Professional Concept)

1️⃣ Why Weighted Mean Exists

Normal mean assumes:

Every value has equal importance.

$$\mu = \frac{x_1 + x_2 + \ldots + x_n}{n}$$

But in real-world data:

- Some data points are **more important**
- Some observations occur **more times**
- Some samples have **higher confidence**

So we generalize mean.

2️⃣ Mathematical Definition

Weighted Mean:

$$\mu = \frac{\sum w_i x_i}{\sum w_i}$$

Where:

- x_i = value
- w_i = weight
- $\sum w_i$ = total weight

---

3 Simple Example (Conceptually)

Suppose:

Marks = [80, 90]
Credits = [3, 1]

Course 1 is more important.

Weighted mean:

$$\mu = \frac{(80 \times 3) + (90 \times 1)}{3 + 1}$$
$$= \frac{240 + 90}{4} = \frac{330}{4} = 82.5$$

Notice:

Normal mean = 85
Weighted mean = 82.5

Because 80 had more weight.

---

4 NumPy Implementation

```
import numpy as np

X = np.array([80, 90])
w = np.array([3, 1])

print(np.average(X, weights=w))
```

Output:

```
82.5
```

---

5 How NumPy Actually Computes It

Internally NumPy does:

```
np.sum(X * w) / np.sum(w)
```

That is exactly the formula.

You could manually verify:

```
print(np.sum(X*w) / np.sum(w))
```

Same result.

---

6 Multi-Dimensional Weighted Average

Now professional level.

Example:

```
X = np.array([[80, 90],
              [70, 60]])

w = np.array([3, 1])
```

If we do:

```
np.average(X, axis=1, weights=w)
```

What happens?

Row 1:
(80×3 + 90×1) / 4 = 82.5

Row 2:
(70×3 + 60×1) / 4 = 67.5

Output:

```
[82.5 67.5]
```

So:

- `axis=1` → weight applied across columns
- weights must match that axis length

---

7 Important Rules (Very Important)

✓ Weights must match the axis length

If axis=1 and there are 2 columns → weights must have length 2.

Otherwise → error.

✓ Weights are NOT required to sum to 1

NumPy automatically divides by total weight.

✓ If no weights given

```
np.average(X)
```

It behaves like `np.mean()`.